Generative AI training processes and resources

Explanatory document / Rahul Niroop / November 8, 2026

A large language model develops through data collection, preparation, model design, pretraining, post-training, evaluation, and deployment. Pretraining adjusts numerical weights to predict tokens. Later instruction and preference training shapes how the model responds to users. Data, compute, energy, and time constrain the work at every stage. Evaluation and monitoring can require earlier stages to be repeated [1-3, 6].

How the stages were sorted

The seven stages are grouped by purpose and output. Collection and preparation produce a documented corpus and token sequences. Design produces a training plan. Pretraining and post-training produce base and assistant weights. Evaluation produces evidence for a release decision, while deployment produces a serving system. This order follows dependencies. Evaluation also runs throughout the lifecycle, so failed checks can send work back to earlier stages [1-3, 6].

Why these resource categories were chosen

The four resource categories answer different questions. Data describes the information used to learn or test. Compute describes processing capacity, memory, and communication infrastructure. Energy describes electricity needed to operate that infrastructure. Time describes elapsed duration and iteration. Keeping compute and time separate prevents aggregate GPU-hours from being mistaken for calendar duration. Skilled labor supports all four categories rather than belonging only to time.

How resource emphasis was assigned

The D, C, E, and T badges identify the characteristic demands of each stage. For example, collection emphasizes data curation and time, while pretraining emphasizes data processing, compute, and continuous energy demand. These overlapping tags are an explanatory interpretation of the workflow, not exclusive categories or measured spending shares. Every stage still uses time, people, and supporting infrastructure.

How the evidence was classified

Each model figure is classified by model version, hardware, scope, and evidence type. Developer-reported GPU-hours are kept separate from derived time and energy estimates. DeepSeek's rental-based dollar estimate is identified as an assumption-based calculation. GPT-4 quantities absent from the cited report remain undisclosed. This preserves uncertainty and prevents unlike measurements from becoming an efficiency ranking [4-7].

Stage and resource relationships

D = data, C = compute, E = energy, T = time. Tags mark emphasis, not measured budget shares. Skilled people support every stage.

Stage	Output	Emphasis	Basis for emphasis
Collect data	Source corpus	D, T	Curation, rights, and coverage
Prepare and tokenize	Token batches and test split	D, C	Filtering, encoding, and leakage control
Design and pilot	Training plan	C, T	Capacity choices and small experiments
Pretrain	Base model weights	D, C, E	Repeated distributed numerical work
Post-train	Assistant model	D, C, T	Demonstrations, feedback, and iteration
Evaluate and improve	Release evidence	D, T	Independent tests and expert review
Deploy and monitor	Serving system and logs	C, E, T	Recurring generation and operation

Model evidence and learning reflection

Examples and cost boundaries

Meta reports 1,720,320 A100 GPU-hours for Llama 2 70B pretraining and 30.84 million H100 GPU-hours for Llama 3.1 405B training [4, 5]. Their pretraining datasets contain 2 trillion and more than 15 trillion tokens, respectively. DeepSeek-V3 reports 14.8 trillion pretraining tokens and 2.788 million H800 GPU-hours across official pretraining, context extension, and post-training. Its $5.576 million estimate assumes $2 per GPU-hour and excludes prior research and ablation experiments [6]. A rental equivalent is not a complete project budget. The GPT-4 report withholds model size, hardware, and training-compute details, so the artifact does not invent comparable figures [7].

Design rationale and portfolio relevance

The numbered flow explains dependencies, resource badges connect the two views, and the return arrow represents iteration. Direct labels and a limited palette preserve meaning without relying on color alone. Proportional bars show disclosed GPU-hour totals with hardware and scope beside each value. The accessible web version supplies selectable text, a matching relationship table, source links, and labeled controls. The first portfolio artifact demonstrates ML classification; this one adds infrastructure analysis and quantitative reasoning.

Reflection on learning

Challenge and strategy

The most challenging distinction in this analysis is between aggregate computation, elapsed time, electricity, and total project cost. A large GPU-hour number can appear to describe all four. I separate each claim into its unit, hardware, scope, and assumptions before comparing it. The DeepSeek example makes this concrete: 2.664 million pretraining GPU-hours becomes about 54.2 days only after specifying 2,048 concurrent GPUs and full occupancy. The calculation is useful precisely because its assumptions remain visible.

Learning and course connections

My main takeaway is that understanding an algorithm also requires understanding the system around it. Next-token prediction supplies a self-supervised training target, while supervised demonstrations and preference feedback shape later behavior. This connects learning objectives with the course concepts of optimization and evaluation. I also see why data quality and model capacity must be considered together under a compute budget. A published dollar amount alone cannot establish that one model is more capable or efficient than another, especially when hardware and training scopes differ.

Application to AI engineering

For AI engineering and infrastructure work, I would define a task and evaluation criteria before choosing a model or estimating hardware needs. I would track data provenance, record the accounting boundary for costs and electricity, and measure latency and quality on the intended workload. I would also plan checkpoints, monitoring, and revision when tests fail. A useful next project would compare smaller models on the same task and hardware, separating measured results from estimates. That would turn the reasoning in this conceptual artifact into evidence from an actual experiment.

Calculation methods and references

Elapsed time

Elapsed-time equivalent = GPU-hours / concurrent GPUs / 24. DeepSeek-V3 pretraining gives 2,664,000 / 2,048 / 24 = 54.2 days at full occupancy. The report says pretraining finished in less than two months [6]. Preparation, research, evaluation, and interruptions extend the overall project.

Energy

Capacity-based proxy = GPU-hours x stated power capacity in kW. Llama 2 70B gives 1,720,320 x 0.400 = 688,128 kWh, or 688 MWh [4]. Llama 3.1 405B gives 30,840,000 x 0.700 = 21,588,000 kWh, or 21.59 GWh [5]. These are not metered electricity measurements. The cards describe power capacity adjusted for power-usage efficiency, so check that boundary before adding facility overhead. Carbon emissions also depend on electricity sources and accounting methods.

Dollar cost

DeepSeek-V3 reports 2.664M pretraining + 0.119M context-extension + 0.005M post-training GPU-hours = 2.788M. At its assumed $2 per H800 GPU-hour, the equivalent cost is $5.576M [6]. Earlier research and ablations are excluded. Staff, data acquisition, infrastructure ownership, and ongoing serving require separate accounting. Rental quotes may already bundle electricity, so adding an energy bill without checking the boundary can double-count costs.

Terminology

A token is a word, word part, or character. Parameters are learned numerical weights. SFT means supervised fine-tuning on demonstrations. RLHF means reinforcement learning from human feedback. GPU means graphics processing unit. TPU means tensor processing unit. Inference generates outputs with trained weights and ordinarily does not update them.

References

[1] Wolfram, S. (2023). What Is ChatGPT Doing and Why Does It Work

[2] IBM (n.d.). What are large language models

[3] Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback

[4] Meta (2023). Llama 2 model card

[5] Meta (2024). Llama 3.1 model card

[6] DeepSeek-AI (2025). DeepSeek-V3 Technical Report, version 2, Table 1

[7] OpenAI (2023). GPT-4 Technical Report, section 2

[8] Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models

[9] IBM Technology (2023). How Large Language Models Work [Video]

[10] Eye on Tech (2023). What is ChatGPT? (In About A Minute) [Video]